

PHYS 4027 / 5027 • FALL 2026 • LECTURE 3

Root Finding



Prof. Alex Chen

Department of Physics

Washington University in St. Louis

Today



- **The problem.** Solve $f(x) = 0$.
- **Iterative methods.** Improving a guess step by step, and how fast the error shrinks.
- **Implementation.** Tolerances, failure, and function as an argument in C++.

Part I · The problem



Find x such that $f(x) = 0$

Why $f(x) = 0$?

- **Orbit periods.** The turning points of an orbit solve $E = V_{\text{eff}}(r)$. Any perturbation to $1/r$ makes this transcendental, and the period integral $\oint dr / \sqrt{E - V_{\text{eff}}(r)}$ cannot even be written down until the roots are in hand.
- **Wave dispersion.** A wave with wavenumber k can only oscillate at frequencies with $D(\omega, k) = 0$. Deriving the dispersion relation is physics; solving it is nontrivial numerics.
- **Phase transitions.** The BCS gap $\Delta(T)$ is defined implicitly by $1 = \lambda \int \tanh(\sqrt{\xi^2 + \Delta^2}/2T) / \sqrt{\xi^2 + \Delta^2} d\xi$: an equation to solve, not a formula to evaluate.

Often $f(x) = 0$ is not the problem itself but a step inside it, solved thousands of times.

When a formula exists



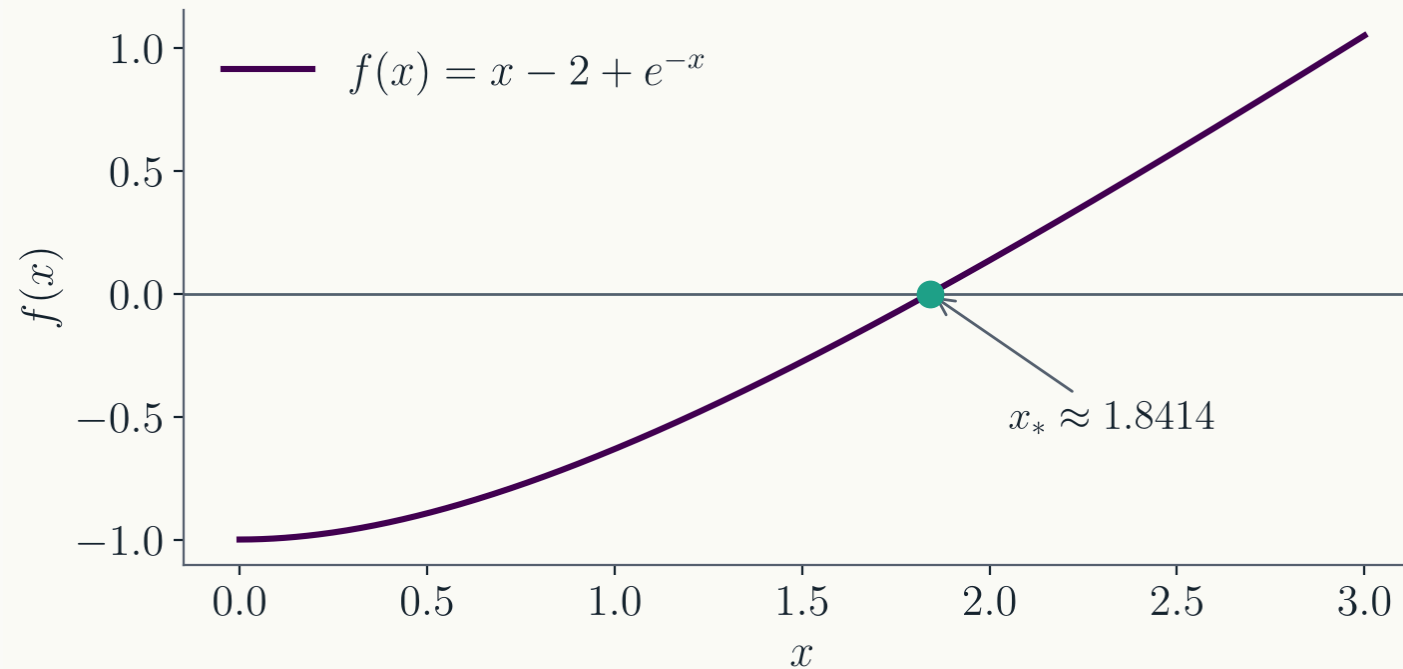
$$ax^2 + bx + c = 0 : \quad x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- When $b^2 \gg 4ac$, one root computes as $-b + \sqrt{b^2 - 4ac}$: two nearly equal numbers subtracted, i.e. catastrophic cancellation (Lecture 1).
- $a = 1, b = 10^8, c = 1$: the small root is -1.0×10^{-8} . The formula gives -7.45×10^{-9} , 25% off.
- The fix avoids the subtraction: with $q = -\frac{1}{2} (b + \text{sgn}(b)\sqrt{b^2 - 4ac})$, take $x_1 = q/a$ and $x_2 = c/q$.

Most equations do not have a formula at all.

When no formula exists

$x = 2 - e^{-x}$, or equivalently $f(x) = x - 2 + e^{-x} = 0$.



There is no closed-form solution. A numerical method instead produces a sequence $x_0, x_1, x_2, \dots$ approaching $x_ = 1.84140566\dots$*

Part II · Iterative methods



Fixed point · Bisection · Newton · Secant

Fixed-point iteration



- Rewrite $f(x) = 0$ in the form $x = g(x)$. For example $g(x) = x - f(x)$, or any other rearrangement.
- Start from a guess x_0 and iterate:

$$x_{n+1} = g(x_n)$$

- Stop when x_{n+1} and x_n agree to the precision you want.
- Our example is already in that form: $g(x) = 2 - e^{-x}$.

x_ is called a fixed point of g : plugging it in gives back x_* .*

Fixed-point iteration



$$x_{n+1} = 2 - e^{-x_n} \text{ from } x_0 = 1:$$

n	x_n	$ x_n - x_* $
0	1.00000	8.4×10^{-1}
1	1.63212	2.1×10^{-1}
2	1.80449	3.7×10^{-2}
3	1.83544	6.0×10^{-3}
4	1.84046	9.5×10^{-4}
5	1.84126	1.5×10^{-4}
6	1.84138	2.4×10^{-5}

- Always need to start from a guess x_0 . We choose $x_0 = 1$ arbitrarily.
- Each step shrinks the error by about the same factor, here 0.16.
- About one more correct digit every 1.3 steps.
- Why 0.16?

Convergence



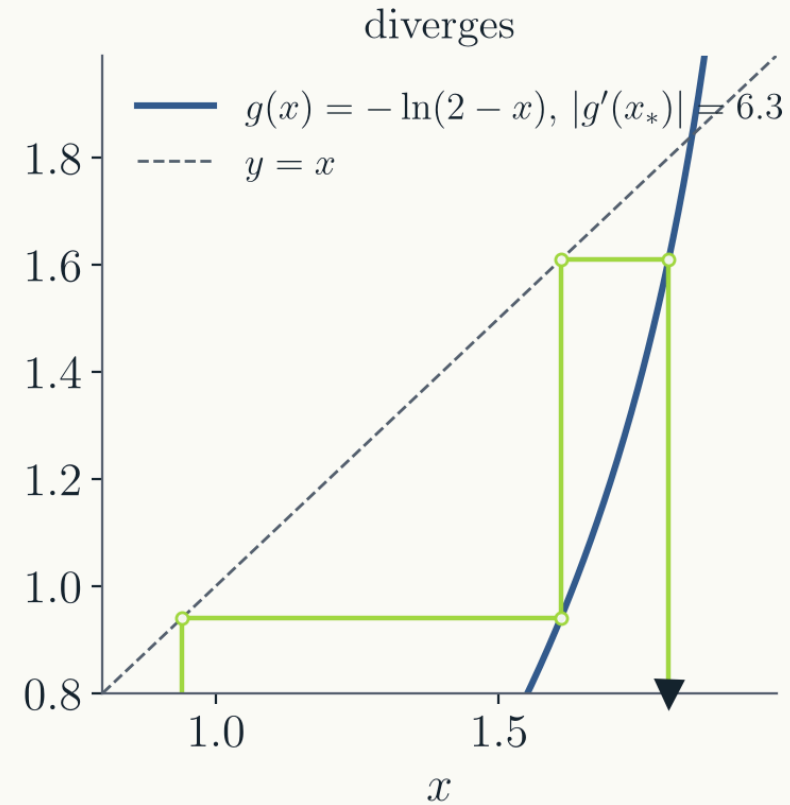
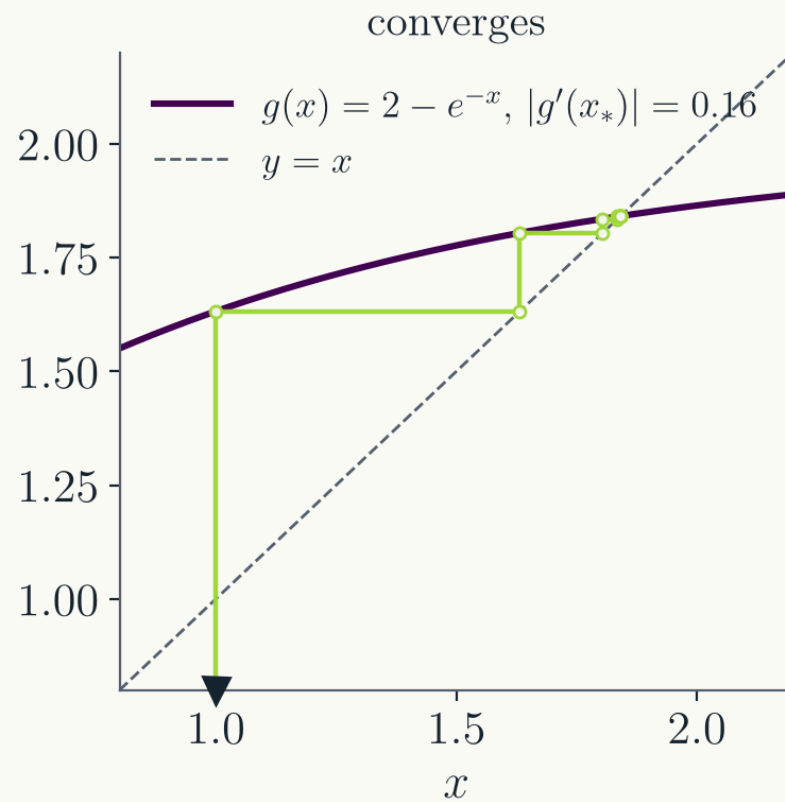
Expand g around the fixed point x_* , where $g(x_*) = x_*$:

$$x_{n+1} - x_* = g(x_n) - g(x_*) = g'(x_*) (x_n - x_*) + \cdots$$

- Each step multiplies the error by $g'(x_*)$. The iteration converges if $|g'(x_*)| < 1$, and then $|x_n - x_*| \approx |g'(x_*)|^n |x_0 - x_*|$.
- This is *linear* convergence: a fixed number of digits per step.
- For $g(x) = 2 - e^{-x}$: $g'(x_*) = e^{-x_*} = 0.159$. That is the 0.16 in the table.

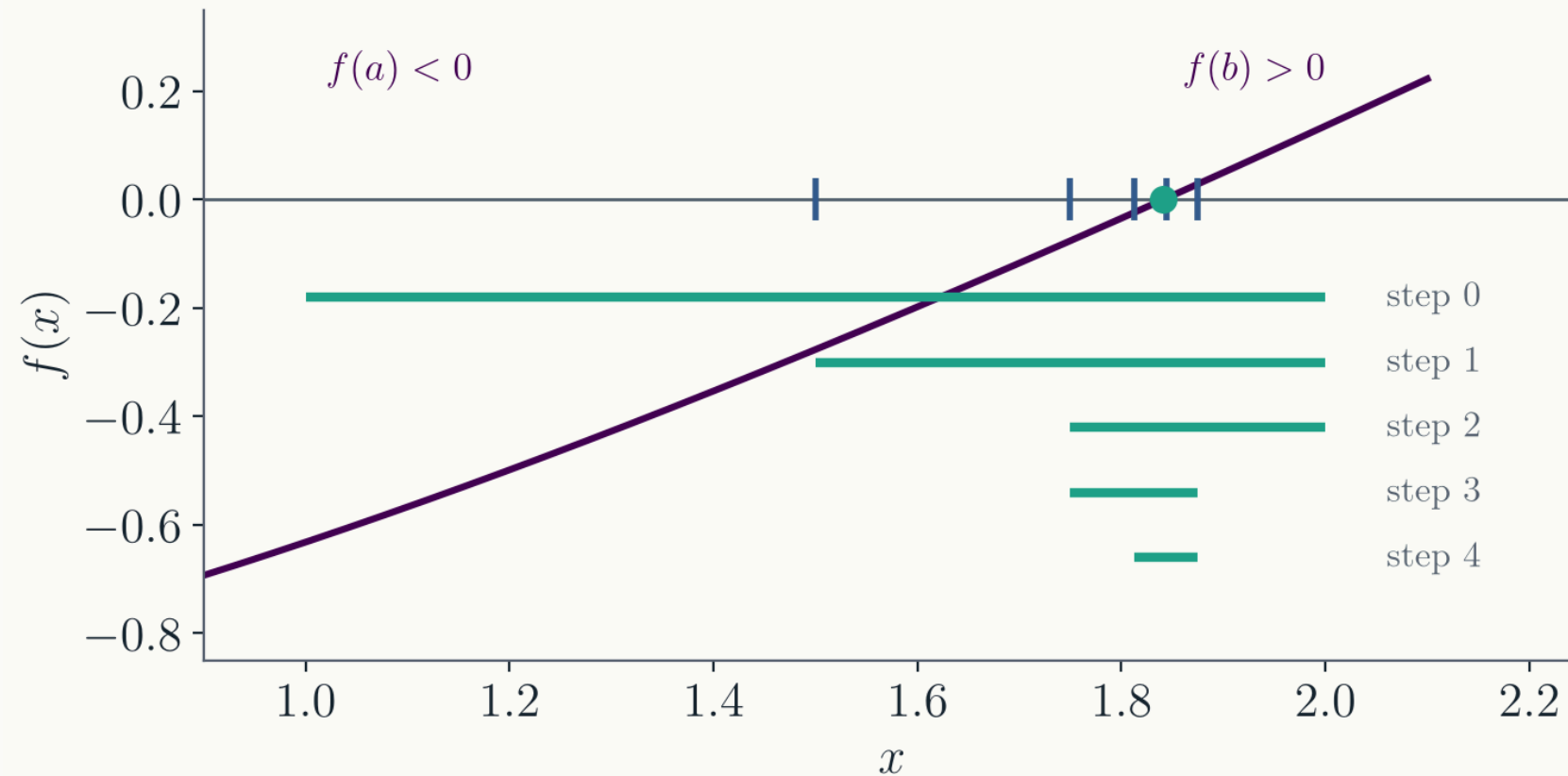
Convergence is slow when $|g'(x_)|$ is close to 1. The iteration will diverge when it exceeds 1.*

When $|g'| > 1$



The same equation rearranged as $x = -\ln(2 - x)$ has $|g'(x_*)| = 6.3$: the iteration runs away. If g does not contract, its inverse does, because $(g^{-1})' = 1/g'$. Solve $x = g^{-1}(x)$ instead.

Bisection method



- Find a, b with $f(a)$ and $f(b)$ of opposite sign. If f is continuous, a root lies between them.
- Take the midpoint. Keep the half where the sign still changes. Repeat.

Bisection method



Start from $[a_0, b_0]$ with $f(a_0) f(b_0) < 0$. For $n = 0, 1, 2, \dots$:

$$x_{n+1} = \frac{1}{2} (a_n + b_n)$$

$$[a_{n+1}, b_{n+1}] = \begin{cases} [a_n, x_{n+1}] & \text{if } f(a_n) f(x_{n+1}) < 0, \\ [x_{n+1}, b_n] & \text{otherwise,} \end{cases}$$

until $b_n - a_n$ (or $|f(x_{n+1})|$) is small enough.

Each update preserves the sign change $f(a_n) f(b_n) < 0$, so the interval contains a root at every step.

Bisection method



- After n steps the interval has width $(b - a)/2^n$. To reach width ε :

$$n = \log_2 \frac{b - a}{\varepsilon}$$

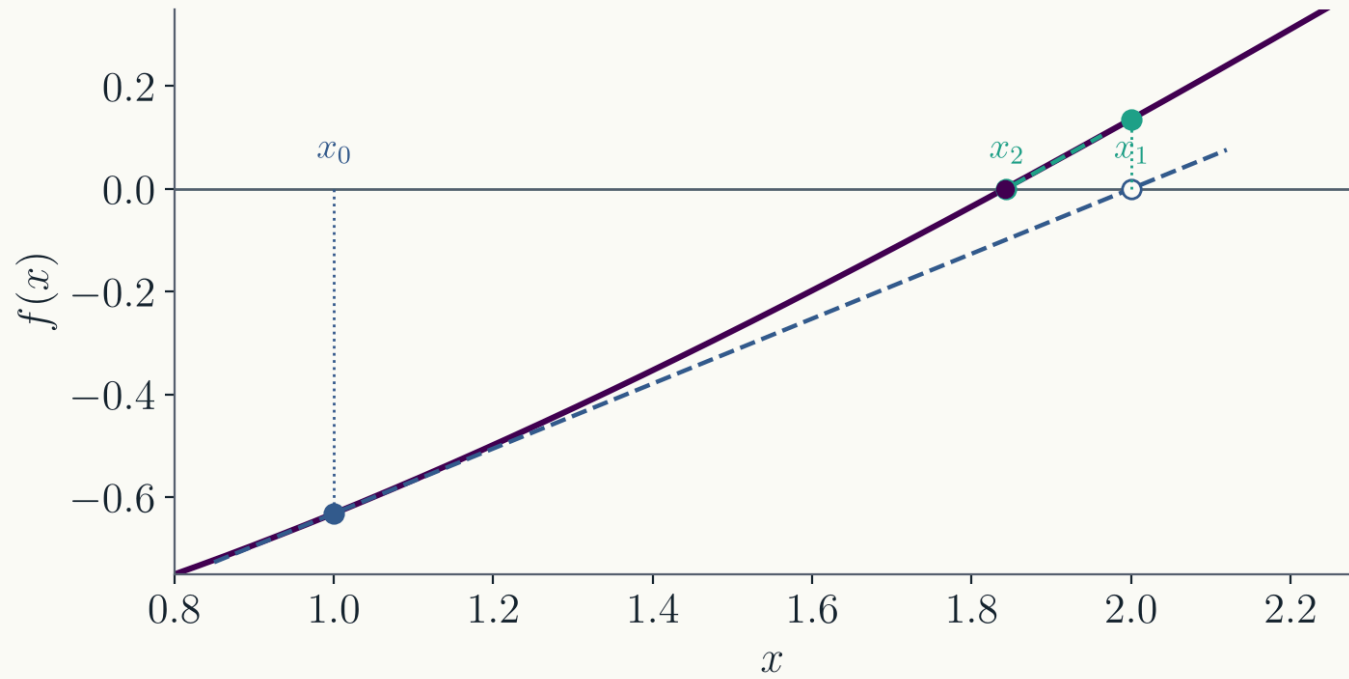
- $[1, 2]$ down to 10^{-15} : 50 steps. Fixed-point iteration at rate 0.16 needed 19.
- The count does not depend on f . With a valid bracket, bisection cannot fail.

Drawbacks of Bisection method

- **No sign change.** $f(x) = x^2$ touches zero without crossing. There is no bracket, so bisection never starts.
- **A sign change that is not a root.** $f(x) = 1/x$ on $[-1, 2]$ changes sign, and bisection converges to $x = 0$, which is a pole, not a root.
- **Several roots in the bracket.** You get one of them, and you do not choose which.

Bisection finds a sign change; only continuity makes that a root.

Newton's method



- Follow the tangent line at x_n down to the axis: $x_{n+1} = x_n - f(x_n)/f'(x_n)$.
- It is fixed-point iteration with $g(x) = x - f(x)/f'(x)$.

Newton's method



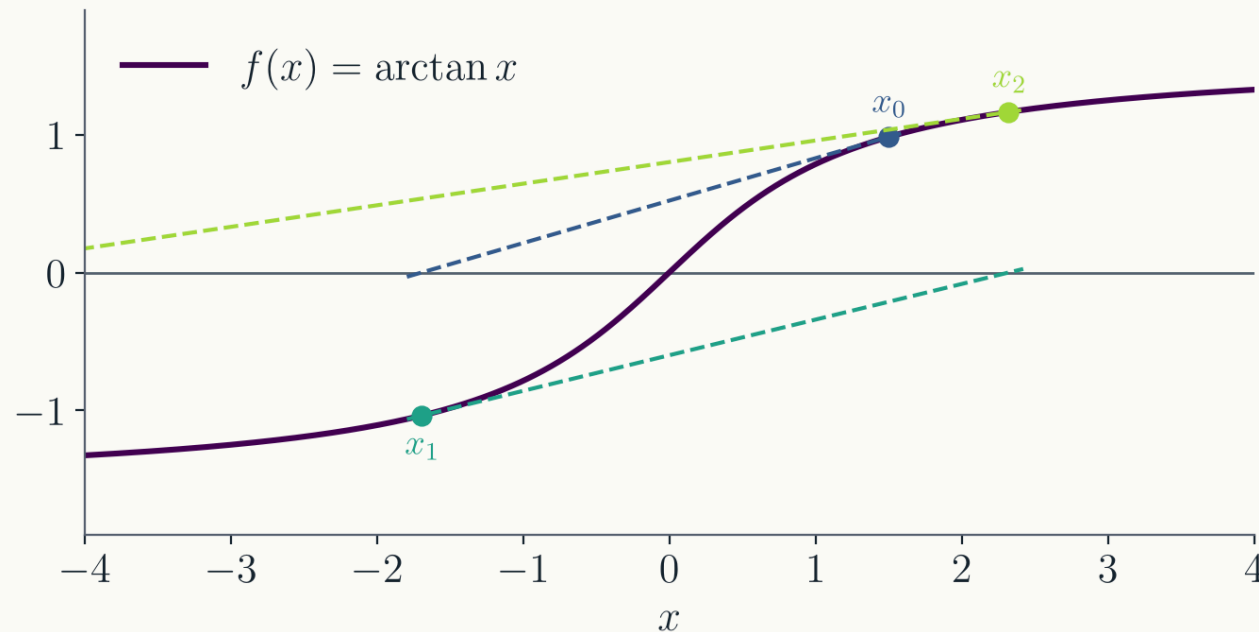
Expand f around x_* to second order and use $f(x_*) = 0$:

$$x_{n+1} - x_* = \frac{f''(x_*)}{2f'(x_*)} (x_n - x_*)^2 + \dots$$

- The error is *squared* each step (quadratic convergence): k correct digits means $e_n \sim 10^{-k}$, and $e_n^2 \sim 10^{-2k}$ means $2k$ correct digits.
- In fixed-point language: $g'(x) = f f'' / (f')^2$, so $g'(x_*) = 0$. The linear term is gone.

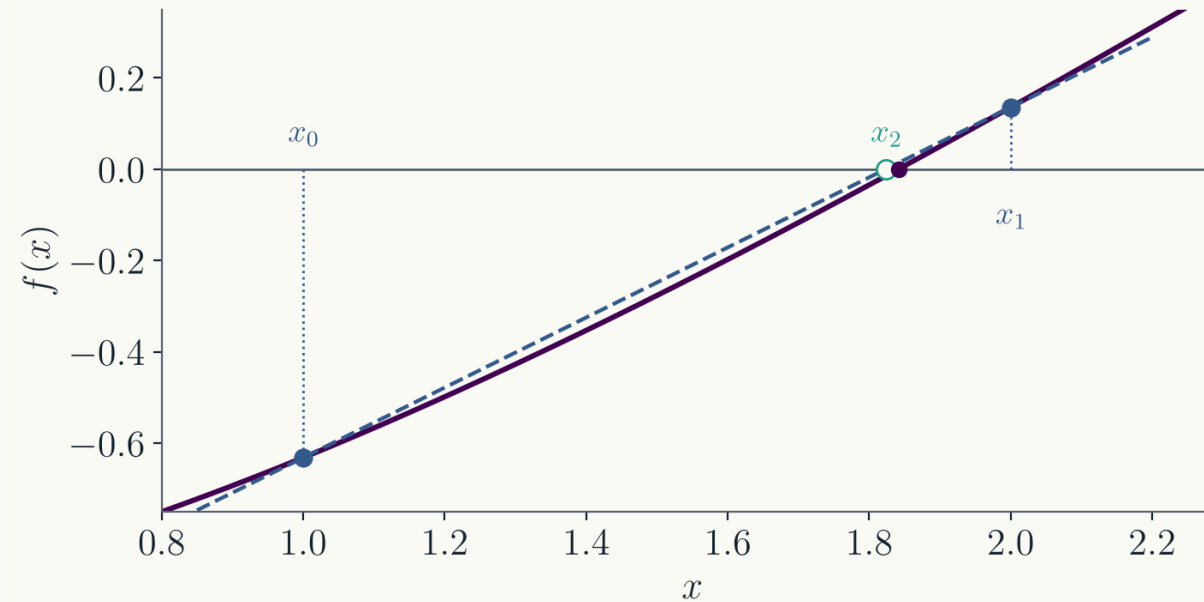
The price: it needs f' , and a starting point close enough to the root.

Drawbacks of Newton's method



- $f(x) = \arctan x$ from $x_0 = 1.5$: each tangent overshoots and lands farther away. From $x_0 = 1.3$ it converges.
- $f'(x_n) \approx 0$ sends x_{n+1} very far away. The iteration can also get stuck in a cycle.
- There is no bracket and no guarantee: fast near a root, unpredictable far from one.

Secant method



- No f' ? Use the slope of the line through the last two points instead of the tangent:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

- Needs two starting points, not a bracket, and only f .

Secant method



- Convergence order is the golden ratio, $\varphi = (1 + \sqrt{5})/2 \approx 1.62$: the error obeys $e_{n+1} \approx C e_n e_{n-1}$.
- Each step costs *one* new evaluation of f . Newton costs f and f' .
- Per function evaluation, secant is usually faster: two secant steps (two evaluations) give order $\varphi^2 \approx 2.6$; one Newton step (two evaluations) gives order 2.
- Shares Newton's failure modes, plus one more: if $f(x_n) = f(x_{n-1})$, the slope is 0/0.

Practical root finders combine a fast method with bisection as a fallback; Brent's method is the standard example.

Comparison

$|x_n - x_*|$ for $f(x) = x - 2 + e^{-x}$. Fixed point and Newton from $x_0 = 1$; bisection on $[1, 2]$; secant from $x_0 = 1, x_1 = 2$.

n	fixed point	bisection	secant	Newton
1	2.1×10^{-1}	9.1×10^{-2}	1.8×10^{-2}	1.6×10^{-1}
2	3.7×10^{-2}	3.4×10^{-2}	2.5×10^{-4}	2.1×10^{-3}
3	6.0×10^{-3}	2.9×10^{-2}	4.2×10^{-7}	4.1×10^{-7}
4	9.5×10^{-4}	2.3×10^{-3}	9.9×10^{-12}	1.6×10^{-14}
5	1.5×10^{-4}	1.3×10^{-2}	0	0

- Needs: a contracting g / a bracket / two starts / f' and a good start.
- Rate: linear (0.16) / linear (0.5, not monotone) / order 1.6 / order 2.

Part III · Implementation



Tolerance, failure, and C++

When to stop

- Three natural tests: the residual $|f(x_n)| < \varepsilon$; the step $|x_{n+1} - x_n| < \varepsilon$; the bracket $b - a < \varepsilon$.
- Absolute or relative? $|x_{n+1} - x_n| < \varepsilon |x_n|$ is the same test for a root at 10^{-3} and at 10^9 . Absolute tolerance only makes sense when you know the scale, or the root may be zero.
- **The floor.** Near $x_* = 1.84$, doubles are 2.2×10^{-16} apart (Lecture 1). Ask for $|x_{n+1} - x_n| < 10^{-20}$ and the loop never ends. The residual has a floor too: about $\epsilon |x_* f'(x_*)|$.

Do not ask for more digits than the format has.

Reporting failure

- Always set a maximum number of iterations; without one, a non-converging iteration loops forever.
- NaN propagates silently: `NaN < tol` is false, so a NaN iterate never triggers the stop test. Check with `std::isnan`.
- Return both the answer and whether the iteration converged:

```
std::tuple<bool, double> result = bisection(f, 1.0, 2.0, 1e-12);  
bool ok = std::get<0>(result);    // std::get<n>: the n-th tuple element  
double x = std::get<1>(result);  
if (!ok) { std::cout << "did not converge\n"; }
```

Passing a function to a function

Root-finding functions take f as an argument. This requires writing a function template:

```
template <typename F>           // F: anything callable as f(x)
double evaluate(const F& f, double x) {
    return f(x);
}

double square(double x) { return x * x; }
evaluate(square, 3.0);           // 9.0
```

The compiler generates a version of `evaluate` for every type `F` it is called with.

Lambda functions



A function can also be written with no name:

```
const double e = 0.3, M = 1.0;
auto kepler = [e, M](double E) { return E - e * std::sin(E) - M; };
evaluate(kepler, 1.0);

evaluate([](double x) { return x * x; }, 3.0); // 9.0
```

- The brackets are the *capture list*: the outside variables the lambda may use.
- This is how a root finder gets an f with parameters in it: capture them.

Where this goes next



- **Next Lecture:** interpolation
- References:
 - Numerical Recipes, 3rd ed., Chapter 9 (root finding).
 - Burden & Faires, *Numerical Analysis*, Chapter 2.
 - 3Blue1Brown, *Newton's fractal* (what Newton does far from a root).